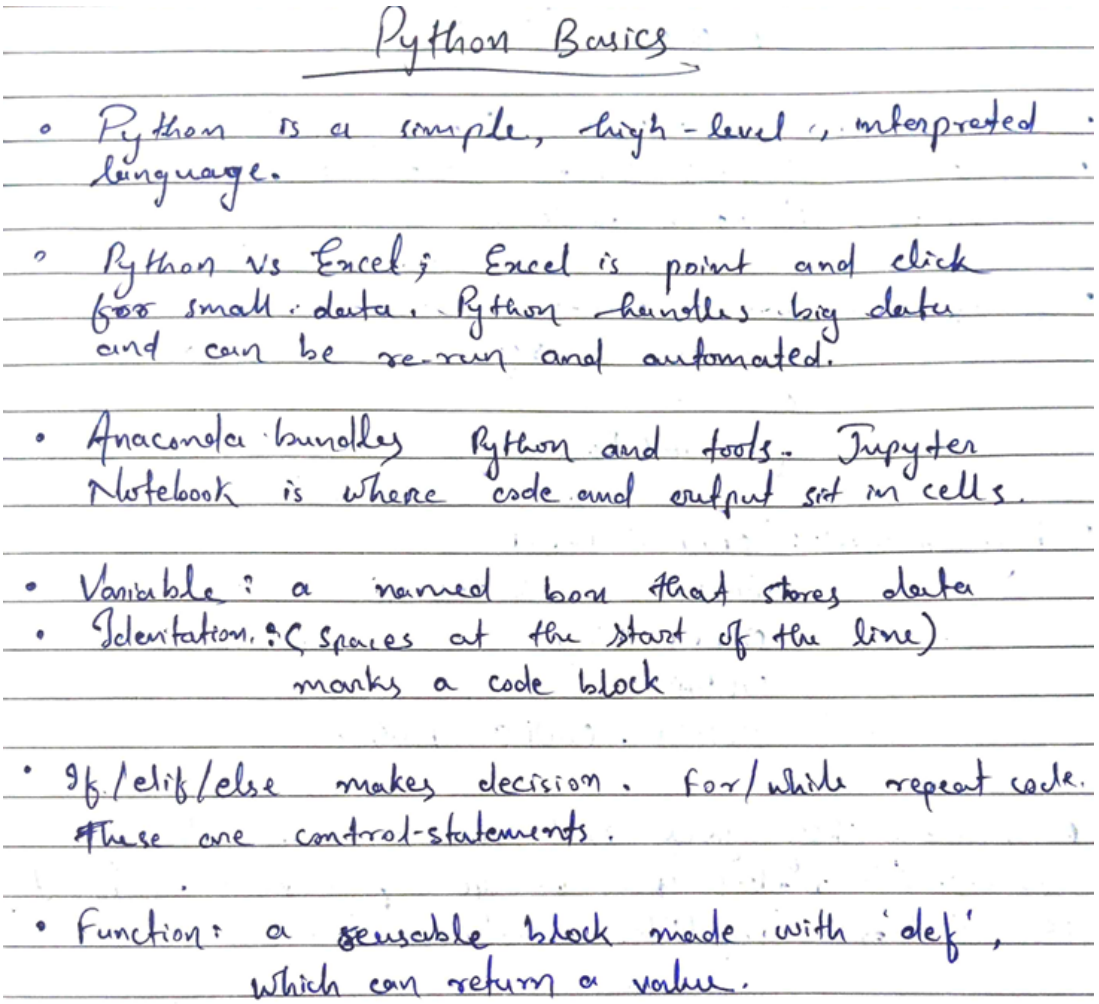
ANALYTICS/COMPUTING WITH PYTHON
SEM-III

CLASS WORK

NAME: Pratyush Pravanjan Pradhan
COURSE: B.Sc. (H) Computer Science
ROLL No. : 251248

UNIT I

Python Basics and Functions



In [1]:
```
nm = "Meera"
ag = 19
pct = 72.5
stu = True

print("Name:", nm)
print("Age:", ag)
print("Percentage:", pct)
print("Is student:", stu)

print(type(nm))
print(type(ag))
print(type(pct))
print(type(stu))
```

```
Name: Meera
Age: 19
Percentage: 72.5
Is student: True
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

In [2]:
```python
x = 17
y = 5
print("Addition:", x + y)
print("Subtraction:", x - y)
print("Multiplication:", x * y)
print("Division:", x / y)
print("Floor division:", x // y)
print("Remainder:", x % y)
print("Power:", x ** y)
```

```
Addition: 22
Subtraction: 12
Multiplication: 85
Division: 3.4
Floor division: 3
Remainder: 2
Power: 1419857
```

In [3]:
```python
sb = ["Java", "Maths", "Stats"]
print("Subjects:", sb)
print("First:", sb[0])
print("Last:", sb[-1])

sb.append("English")
print("After adding:", sb)
print("Count:", len(sb))

mk = {"Java": 67, "Maths": 91, "Stats": 58}
print("Marks:", mk)
print("Maths marks:", mk["Maths"])

mk["English"] = 84
print("After adding English:", mk)
```

```
Subjects: ['Java', 'Maths', 'Stats']
First: Java
Last: Stats
After adding: ['Java', 'Maths', 'Stats', 'English']
Count: 4
Marks: {'Java': 67, 'Maths': 91, 'Stats': 58}
Maths marks: 91
After adding English: {'Java': 67, 'Maths': 91, 'Stats': 58, 'English
': 84}
```

In [4]:
```python
sc = 62
if sc >= 80:
    print("Grade A")
elif sc >= 60:
    print("Grade B")
```

```
else:
    print("Grade C")

for s in sb:
    print(s)

for i in range(1, 6):
    print(i)

c = 3
while c > 0:
    print("Countdown:", c)
    c = c - 1
print("Done!")
```

```
Grade B
Java
Maths
Stats
English
1
2
3
4
5
Countdown: 3
Countdown: 2
Countdown: 1
Done!
```

In [5]:

```
def hello(p):
    print("Hello", p)

hello("Meera")

def add(a, b):
    return a + b

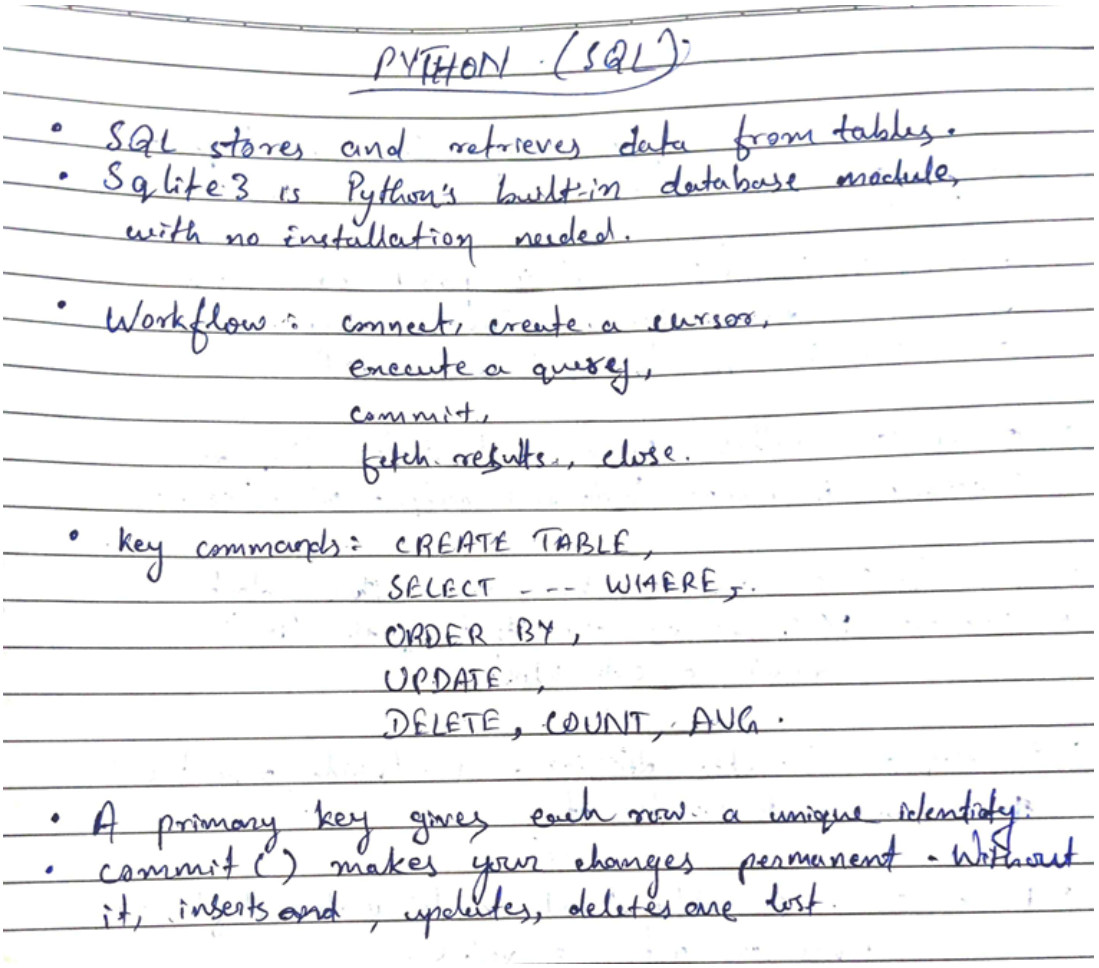
print("8 + 6 =", add(8, 6))

def grade(sc):
    if sc >= 80:
        return "A"
    elif sc >= 60:
        return "B"
    else:
        return "C"

for s in mk:
    print(s, "->", mk[s], "-> Grade", grade(mk[s]))
```

```
Hello Meera
8 + 6 = 14
Java -> 67 -> Grade B
Maths -> 91 -> Grade A
Stats -> 58 -> Grade C
English -> 84 -> Grade A
```

Python and SQL Database



In [6]:
```
import sqlite3

con = sqlite3.connect("lib.db")
cur = con.cursor()
print("Database connected!")

cur.execute("DROP TABLE IF EXISTS books")
cur.execute("CREATE TABLE books (id INTEGER PRIMARY KEY, title TEXT,
print("Table created!")
```

```
Database connected!
Table created!
```

In [7]:
```
cur.execute("INSERT INTO books (title, genre, price) VALUES ('Dune',
cur.execute("INSERT INTO books (title, genre, price) VALUES ('Emma',
cur.execute("INSERT INTO books (title, genre, price) VALUES ('Heat',
cur.execute("INSERT INTO books (title, genre, price) VALUES ('Gone',
cur.execute("INSERT INTO books (title, genre, price) VALUES ('Ulysses
cur.execute("INSERT INTO books (title, genre, price) VALUES ('Cosmos'
con.commit()
print("6 rows inserted!")

cur.execute("SELECT * FROM books")
print("All books:")
for r in cur.fetchall():
    print(r)
```

```
6 rows inserted!
All books:
(1, 'Dune', 'SciFi', 350)
(2, 'Emma', 'Classic', 220)
(3, 'Heat', 'Thriller', 410)
(4, 'Gone', 'Thriller', 300)
(5, 'Ulysses', 'Classic', 180)
(6, 'Cosmos', 'Science', 275)
```

In [8]:
```
cur.execute("SELECT title, price FROM books WHERE price > 250")
print("Books above 250:")
for r in cur.fetchall():
    print(r)

cur.execute("SELECT title, price FROM books ORDER BY price DESC")
print("Highest to lowest price:")
for r in cur.fetchall():
    print(r)
```

```
Books above 250:
('Dune', 350)
('Heat', 410)
('Gone', 300)
('Cosmos', 275)
Highest to lowest price:
('Heat', 410)
('Dune', 350)
('Gone', 300)
('Cosmos', 275)
('Emma', 220)
('Ulysses', 180)
```

In [9]:
```
cur.execute("UPDATE books SET price = 250 WHERE title = 'Emma'")
con.commit()
print("Emma price updated to 250")

cur.execute("DELETE FROM books WHERE title = 'Ulysses'")
con.commit()
print("Ulysses deleted")

cur.execute("SELECT COUNT(*) FROM books")
print("Total books:", cur.fetchone()[0])

cur.execute("SELECT AVG(price) FROM books")
```

```
print("Average price:", cur.fetchone()[0])

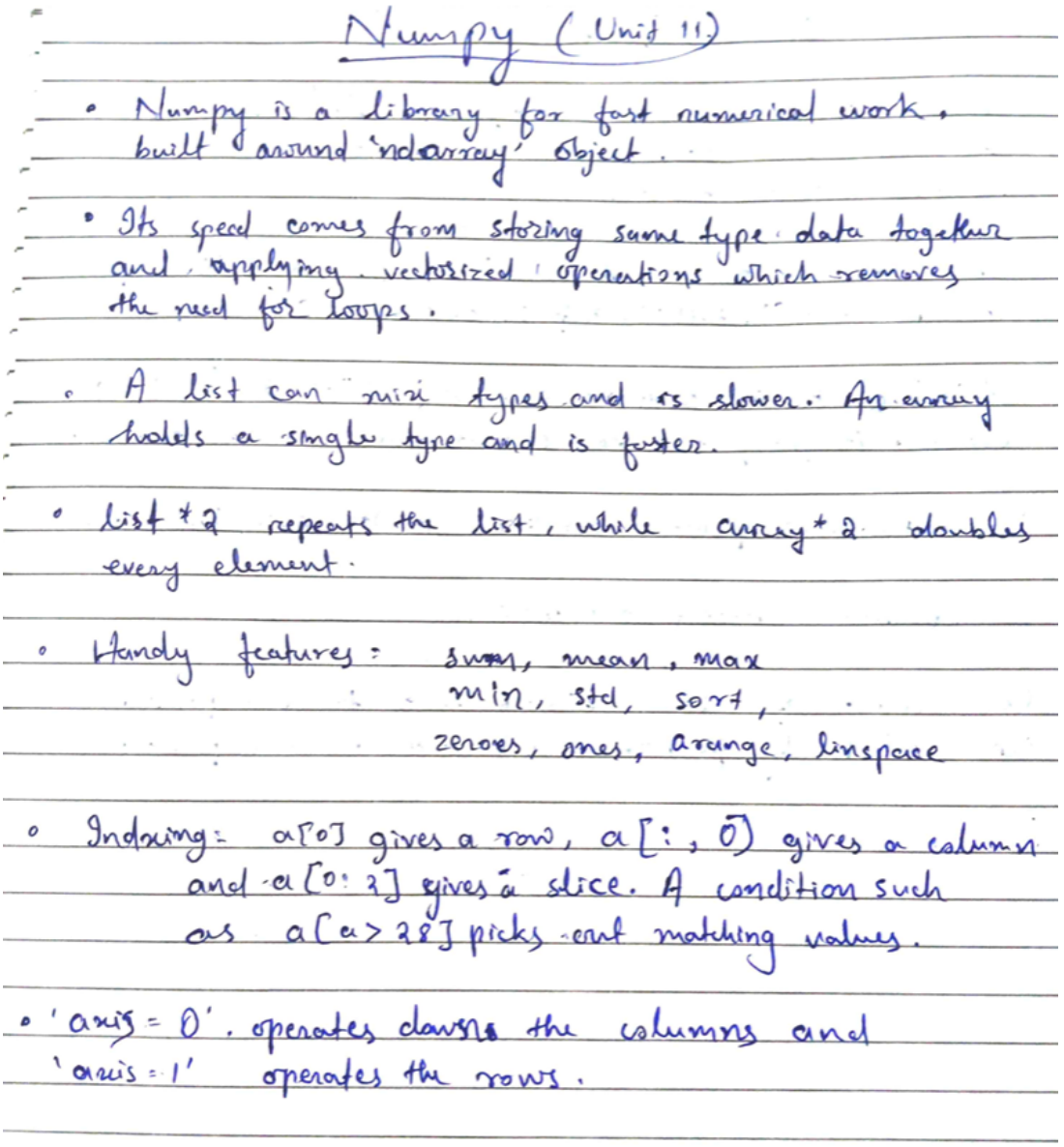
cur.execute("SELECT * FROM books")
print("Final table:")
for r in cur.fetchall():
    print(r)

con.close()
print("Connection closed.")
```

```
Emma price updated to 250
Ulysses deleted
Total books: 5
Average price: 317.0
Final table:
(1, 'Dune', 'SciFi', 350)
(2, 'Emma', 'Classic', 250)
(3, 'Heat', 'Thriller', 410)
(4, 'Gone', 'Thriller', 300)
(6, 'Cosmos', 'Science', 275)
Connection closed.
```

UNIT II

NumPy Arrays, Indexing and Speed



Numpy (Unit II)

- Numpy is a library for fast numerical work, built around 'ndarray' object.
- Its speed comes from storing same type data together and applying vectorized operations which removes the need for loops.
- A list can mix types and is slower. An array holds a single type and is faster.
- list * 2 repeats the list, while array * 2 doubles every element.
- Handy features: sum, mean, max, min, std, sort, zeroes, ones, arange, linspace
- Indexing: a[0] gives a row, a[:, 0] gives a column and a[0:2] gives a slice. A condition such as a[a > 28] picks out matching values.
- 'axis = 0' operates down the columns and 'axis = 1' operates the rows.

In [ ]:
```
import numpy as np
import time

tp = np.array([22, 31, 27, 35, 29, 24])
print("Array:", tp)
print("Type:", type(tp))
print("Shape:", tp.shape)
print("Dtype:", tp.dtype)

print("Sum:", tp.sum())
print("Mean:", tp.mean())
print("Max:", tp.max())
print("Min:", tp.min())
print("Std:", tp.std())
```

```
print("Sorted:", np.sort(tp))

print("tp + 5:", tp + 5)
print("tp * 2:", tp * 2)
print("tp / 10:", tp / 10)
```

```
Array: [22 31 27 35 29 24]
Type: <class 'numpy.ndarray'>
Shape: (6,)
Dtype: int64
Sum: 168
Mean: 28.0
Max: 35
Min: 22
Std: 4.320493798938574
Sorted: [22 24 27 29 31 35]
tp + 5: [27 36 32 40 34 29]
tp * 2: [44 62 54 70 58 48]
tp / 10: [2.2 3.1 2.7 3.5 2.9 2.4]
```

In [ ]:
```
print("Zeros:", np.zeros(4))
print("Ones:", np.ones(4))
print("arange 0 to 9:", np.arange(0, 10))
print("arange step 3:", np.arange(0, 12, 3))
print("linspace:", np.linspace(0, 1, 5))

sl = np.array([[12, 18, 15],
               [20, 10, 25],
               [30, 22, 17]])
print(sl)
print("Shape:", sl.shape)
```

```
Zeros: [0. 0. 0. 0.]
Ones: [1. 1. 1. 1.]
arange 0 to 9: [0 1 2 3 4 5 6 7 8 9]
arange step 3: [0 3 6 9]
linspace: [0.   0.25 0.5  0.75 1.  ]
[[12 18 15]
 [20 10 25]
 [30 22 17]]
Shape: (3, 3)
```

In [ ]:
```
print("First row:", sl[0])
print("Element [1, 2]:", sl[1, 2])
print("First column:", sl[:, 0])
print("First two rows:")
print(sl[0:2])

print("Row-wise mean (axis=1):", sl.mean(axis=1))
print("Column-wise mean (axis=0):", sl.mean(axis=0))

print("Above 28:", tp[tp > 28])
print("Boolean mask:", tp > 28)
```

```
First row: [12 18 15]
Element [1, 2]: 25
First column: [12 20 30]
First two rows:
[[12 18 15]
 [20 10 25]]
Row-wise mean (axis=1): [15.         18.33333333 23.        ]
Column-wise mean (axis=0): [20.66666667 16.66666667 19.        ]
Above 28: [31 35 29]
Boolean mask: [False  True False  True  True False]
```

In [ ]:
```python
bl = list(range(1000000))
ba = np.arange(1000000)

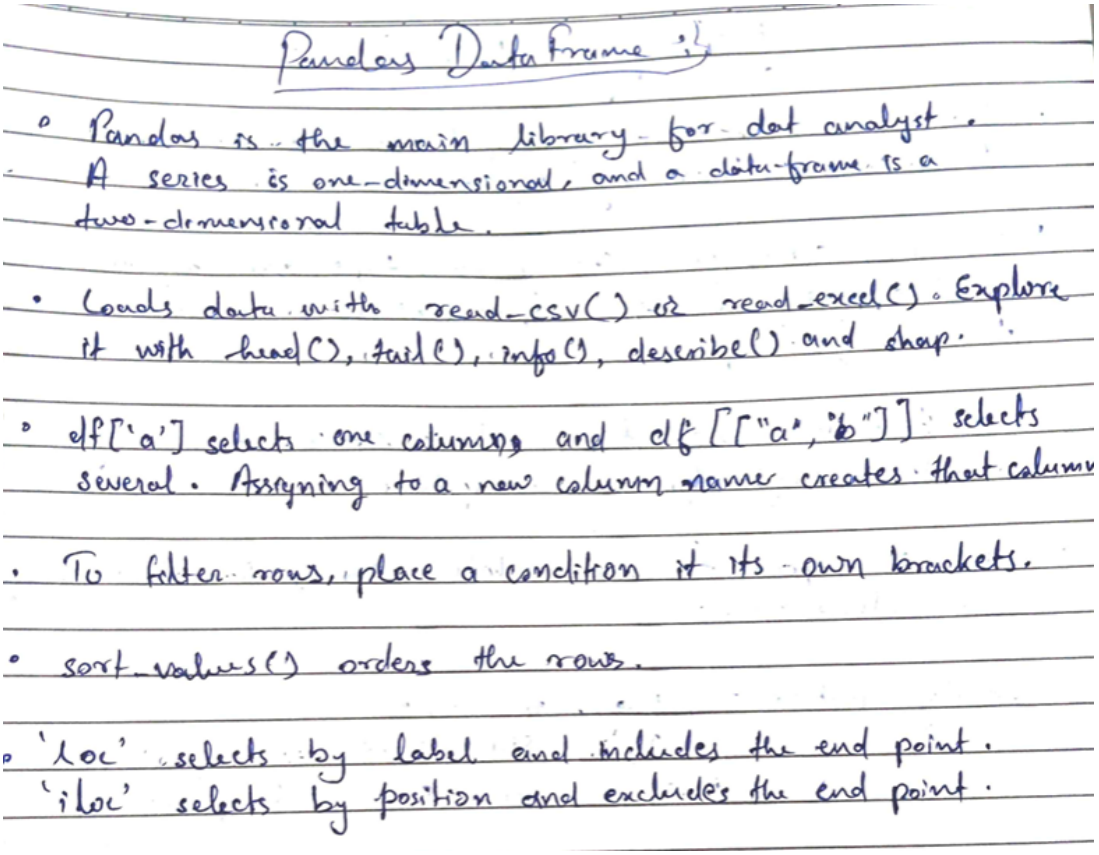
s = time.time()
nl = []
for v in bl:
    nl.append(v * 2)
lt = time.time() - s
print("List time:", round(lt, 4), "seconds")

s = time.time()
na = ba * 2
nt = time.time() - s
print("NumPy time:", round(nt, 4), "seconds")

print("NumPy is about", round(lt / nt), "times faster")
```

```
List time: 0.0623 seconds
NumPy time: 0.0025 seconds
NumPy is about 25 times faster
```

Pandas DataFrame: Import, Filter, loc and iloc



In [ ]:
```
import pandas as pd

d = {
    "name": ["Arun", "Bela", "Chet", "Diya", "Esha", "Faiz"],
    "dept": ["HR", "IT", "IT", "HR", "Ops", "IT"],
    "exp":  [2, 6, 4, 8, 3, 5],
    "sal":  [30, 55, 48, 60, 35, 52]
}
df = pd.DataFrame(d)
print(df)

df.to_csv("emp.csv", index=False)
df = pd.read_csv("emp.csv")
print("Data loaded from CSV")
```

```
   name dept  exp  sal
0  Arun   HR    2   30
1  Bela   IT    6   55
2  Chet   IT    4   48
3  Diya   HR    8   60
4  Esha  Ops    3   35
5  Faiz   IT    5   52
Data loaded from CSV
```

In [ ]:
```
print(df.head(3))
print(df.tail(2))
print("Shape:", df.shape)
print("Columns:", list(df.columns))
df.info()
print(df.describe())

print(df["name"])
```

```
print(df[["name", "sal"]])
```

```
   name dept  exp  sal
0  Arun   HR    2   30
1  Bela   IT    6   55
2  Chet   IT    4   48
   name dept  exp  sal
4  Esha  Ops    3   35
5  Faiz   IT    5   52
Shape: (6, 4)
Columns: ['name', 'dept', 'exp', 'sal']
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   name    6 non-null      str
 1   dept    6 non-null      str
 2   exp     6 non-null      int64
 3   sal     6 non-null      int64
dtypes: int64(2), str(2)
memory usage: 324.0 bytes
            exp        sal
count  6.000000   6.000000
mean   4.666667  46.666667
std    2.160247  11.758685
min    2.000000  30.000000
25%    3.250000  38.250000
50%    4.500000  50.000000
75%    5.750000  54.250000
max    8.000000  60.000000
0    Arun
1    Bela
2    Chet
3    Diya
4    Esha
5    Faiz
Name: name, dtype: str
   name  sal
0  Arun   30
1  Bela   55
2  Chet   48
3  Diya   60
4  Esha   35
5  Faiz   52
```

In [ ]:

```
df["bonus"] = df["sal"] * 0.1
df["lvl"] = "Junior"
df.loc[df["exp"] >= 5, "lvl"] = "Senior"
print(df)

print(df[df["sal"] > 50])
print(df[df["dept"] == "IT"])
print(df[(df["dept"] == "IT") & (df["exp"] > 4)])
print(df[(df["dept"] == "HR") | (df["dept"] == "Ops")])

print(df.sort_values("sal", ascending=False))
```

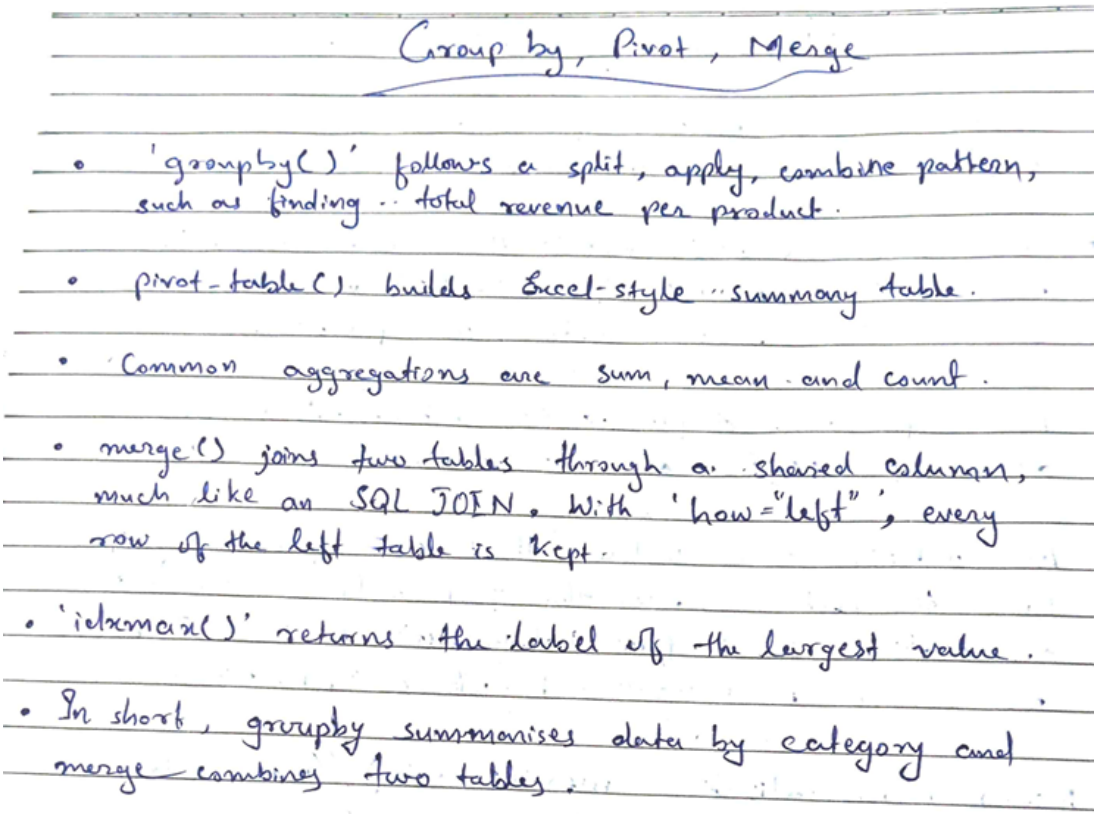
```
   name dept  exp  sal  bonus     lvl
0  Arun   HR    2   30    3.0  Junior
1  Bela   IT    6   55    5.5  Senior
2  Chet   IT    4   48    4.8  Junior
3  Diya   HR    8   60    6.0  Senior
4  Esha  Ops    3   35    3.5  Junior
5  Faiz   IT    5   52    5.2  Senior
   name dept  exp  sal  bonus     lvl
1  Bela   IT    6   55    5.5  Senior
3  Diya   HR    8   60    6.0  Senior
5  Faiz   IT    5   52    5.2  Senior
   name dept  exp  sal  bonus     lvl
1  Bela   IT    6   55    5.5  Senior
2  Chet   IT    4   48    4.8  Junior
5  Faiz   IT    5   52    5.2  Senior
   name dept  exp  sal  bonus     lvl
1  Bela   IT    6   55    5.5  Senior
5  Faiz   IT    5   52    5.2  Senior
   name dept  exp  sal  bonus     lvl
0  Arun   HR    2   30    3.0  Junior
3  Diya   HR    8   60    6.0  Senior
4  Esha  Ops    3   35    3.5  Junior
   name dept  exp  sal  bonus     lvl
3  Diya   HR    8   60    6.0  Senior
1  Bela   IT    6   55    5.5  Senior
5  Faiz   IT    5   52    5.2  Senior
2  Chet   IT    4   48    4.8  Junior
4  Esha  Ops    3   35    3.5  Junior
0  Arun   HR    2   30    3.0  Junior
```

In [ ]:
```python
print("Name at row 2:", df.loc[2, "name"])
print(df.loc[0:2, ["name", "sal"]])
print(df.iloc[0:2, 0:3])

print("Average salary:", df["sal"].mean())
print("Max experience:", df["exp"].max())
print(df["dept"].value_counts())
```

```
Name at row 2: Chet
   name  sal
0  Arun   30
1  Bela   55
2  Chet   48
   name dept  exp
0  Arun   HR    2
1  Bela   IT    6
Average salary: 46.666666666666664
Max experience: 8
dept
IT     3
HR     2
Ops    1
Name: count, dtype: int64
```

Pandas GroupBy, Pivot and Merge



In [ ]:
```
od = pd.DataFrame({
    "prod": ["Tea", "Jam", "Tea", "Rice", "Jam", "Rice"],
    "zone": ["East", "West", "East", "West", "East", "West"],
    "qty":  [20, 15, 30, 10, 25, 40],
    "rate": [5, 8, 5, 12, 8, 12]
})
od["rev"] = od["qty"] * od["rate"]
print(od)

print(od.groupby("prod")["rev"].sum())
print(od.groupby("prod")["rev"].mean())
print(od.groupby("prod")["rev"].count())
print(od.groupby("zone")["qty"].sum())
print(od.groupby("zone")["rev"].sum())
```

```
   prod  zone  qty  rate  rev
0   Tea  East   20     5  100
1   Jam  West   15     8  120
2   Tea  East   30     5  150
3  Rice  West   10    12  120
4   Jam  East   25     8  200
5  Rice  West   40    12  480
prod
Jam     320
Rice    600
Tea     250
Name: rev, dtype: int64
prod
Jam     160.0
Rice    300.0
Tea     125.0
Name: rev, dtype: float64
prod
Jam     2
Rice    2
Tea     2
Name: rev, dtype: int64
zone
East    75
West    65
Name: qty, dtype: int64
zone
East    450
West    720
Name: rev, dtype: int64
```

In [ ]:

```
pv = od.pivot_table(values="rev", index="prod", columns="zone", aggfu
print(pv)

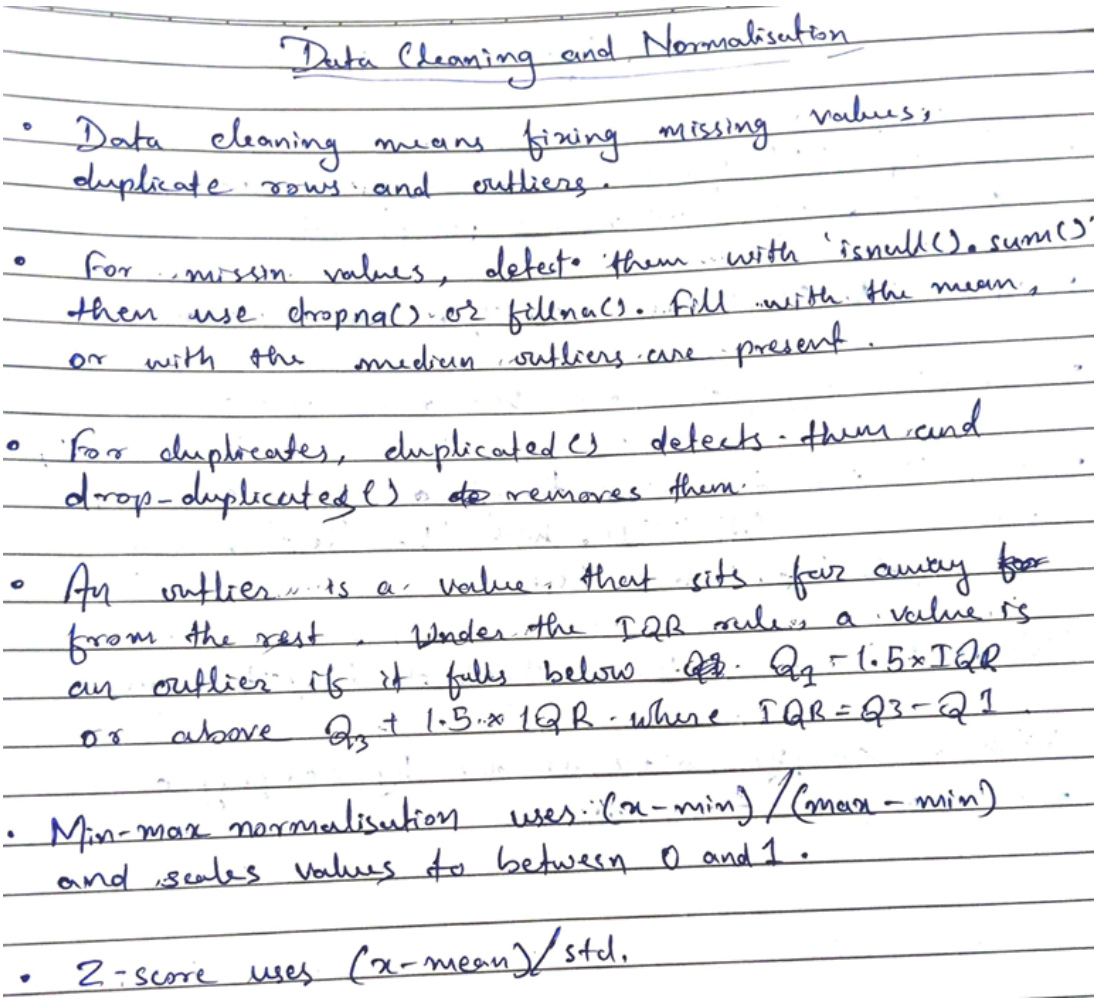
inf = pd.DataFrame({
    "prod": ["Tea", "Jam", "Rice"],
    "typ":  ["Drink", "Food", "Food"]
})
mg = od.merge(inf, on="prod", how="left")
print(mg)

print(mg.groupby("typ")["rev"].sum())

pr = od.groupby("prod")["rev"].sum()
print("Best product:", pr.idxmax())
print("Its revenue:", pr.max())
```

```
zone  East  West
prod
Jam    200   120
Rice     0   600
Tea    250     0
   prod  zone  qty  rate  rev    typ
0   Tea  East   20     5  100  Drink
1   Jam  West   15     8  120   Food
2   Tea  East   30     5  150  Drink
3  Rice  West   10    12  120   Food
4   Jam  East   25     8  200   Food
5  Rice  West   40    12  480   Food
typ
Drink    250
Food     920
Name: rev, dtype: int64
Best product: Rice
Its revenue: 600
```

Data Cleaning, Outliers and Normalization



In [ ]:
```
import numpy as np

d = {
    "name": ["Om", "Pia", "Raj", "Pia", "Sam", "Tara"],
```

```
    "age":  [21, 22, np.nan, 22, 23, 21],
    "sal":  [23000, 26000, 24000, 26000, np.nan, 500000]
}
df = pd.DataFrame(d)
print(df)

print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
df = df.drop_duplicates()
print(df)

df["age"] = df["age"].fillna(df["age"].mean())
df["sal"] = df["sal"].fillna(df["sal"].median())
print(df)
print("Missing left:", df.isnull().sum().sum())
```

```
   name   age       sal
0    Om  21.0   23000.0
1   Pia  22.0   26000.0
2   Raj   NaN   24000.0
3   Pia  22.0   26000.0
4   Sam  23.0       NaN
5  Tara  21.0  500000.0
name    0
age     1
sal     1
dtype: int64
Duplicates: 1
   name   age       sal
0    Om  21.0   23000.0
1   Pia  22.0   26000.0
2   Raj   NaN   24000.0
4   Sam  23.0       NaN
5  Tara  21.0  500000.0
   name    age       sal
0    Om  21.00   23000.0
1   Pia  22.00   26000.0
2   Raj  21.75   24000.0
4   Sam  23.00   25000.0
5  Tara  21.00  500000.0
Missing left: 0
```

In [ ]:
```
q1 = df["sal"].quantile(0.25)
q3 = df["sal"].quantile(0.75)
iqr = q3 - q1
lo = q1 - 1.5 * iqr
hi = q3 + 1.5 * iqr
print("Q1:", q1, "Q3:", q3)
print("Lower:", lo, "Upper:", hi)

out = df[(df["sal"] < lo) | (df["sal"] > hi)]
print(out)

cl = df[(df["sal"] >= lo) & (df["sal"] <= hi)].copy()
print(cl)
```

```
Q1: 24000.0 Q3: 26000.0
Lower: 21000.0 Upper: 29000.0
   name   age       sal
5  Tara  21.0  500000.0
  name    age      sal
0   Om  21.00  23000.0
1  Pia  22.00  26000.0
2  Raj  21.75  24000.0
4  Sam  23.00  25000.0
```

In [ ]:
```
mn = cl["sal"].min()
mx = cl["sal"].max()
cl["mm"] = (cl["sal"] - mn) / (mx - mn)

mu = cl["sal"].mean()
sd = cl["sal"].std()
cl["z"] = (cl["sal"] - mu) / sd

print(cl.round(2))
```

```
  name    age      sal    mm     z
0   Om  21.00  23000.0  0.00 -1.16
1  Pia  22.00  26000.0  1.00  1.16
2  Raj  21.75  24000.0  0.33 -0.39
4  Sam  23.00  25000.0  0.67  0.39
```

Data Visualization

Data Visualisation

- Matplotlib is the foundation library. Seaborn is built on top of it and produces better-looking charts.

Chart	Best for
Bar	Comparing categories
Line	Showing a trend over time
Histogram	Showing the distribution of one variable
Scatter	Showing the relation between two tables
Pie	parts of a whole
Box Plot	Spread and outliers
Heatmap	Correlation

- In a box plot, the box covers the middle 50% of the data and the line inside the median. Dots beyond the whiskers are outliers.

- Correlation runs from -1 to +1. A value near +1 means both variables rise together and value near -1 means one rises while other falls.

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns

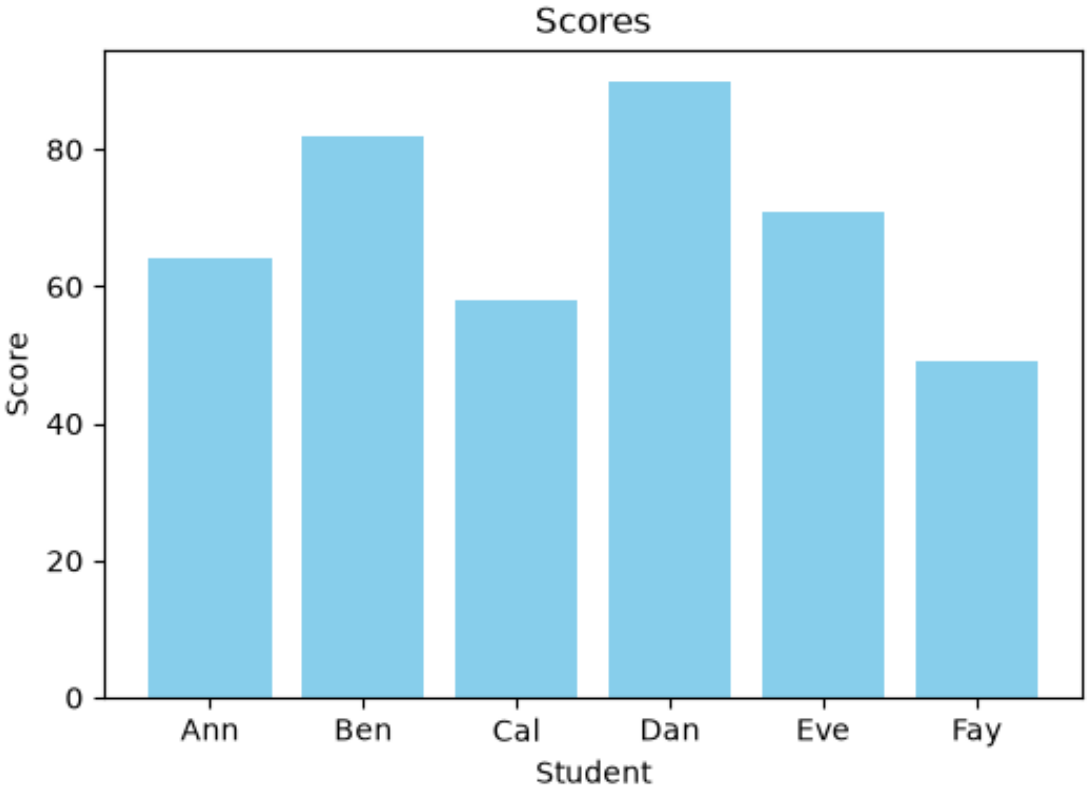
df = pd.DataFrame({
    "tm": ["Red", "Blue", "Red", "Green", "Blue", "Green"],
    "sc": [64, 82, 58, 90, 71, 49],
    "hr": [3, 6, 2, 7, 4, 1],
    "sl": [8, 6, 8, 5, 7, 9]
}, index=["Ann", "Ben", "Cal", "Dan", "Eve", "Fay"])
print(df)

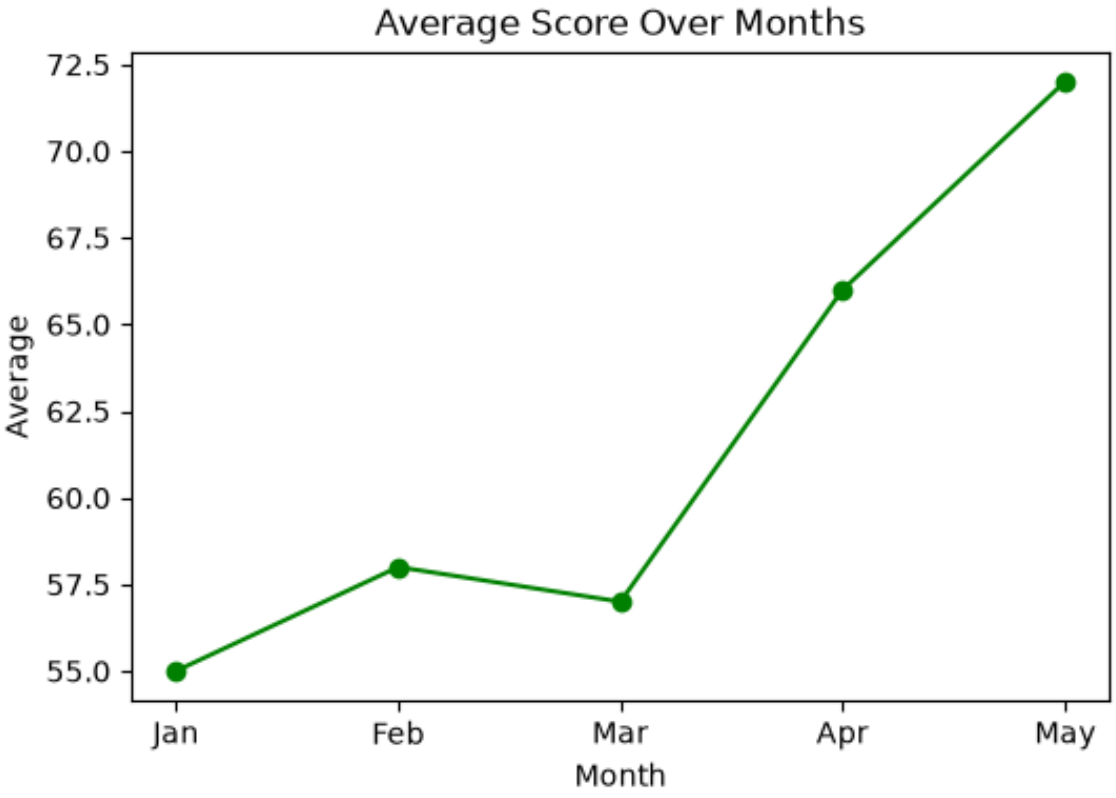
plt.figure(figsize=(6, 4))
plt.bar(df.index, df["sc"], color="skyblue")
plt.title("Scores")
plt.xlabel("Student")
plt.ylabel("Score")
plt.show()

mo = ["Jan", "Feb", "Mar", "Apr", "May"]
av = [55, 58, 57, 66, 72]
plt.figure(figsize=(6, 4))
plt.plot(mo, av, marker="o", color="green")
plt.title("Average Score Over Months")
plt.xlabel("Month")
plt.ylabel("Average")
```

```
plt.show()
```

```
        tm  sc  hr  sl
Ann    Red  64   3   8
Ben   Blue  82   6   6
Cal    Red  58   2   8
Dan  Green  90   7   5
Eve   Blue  71   4   7
Fay  Green  49   1   9
```

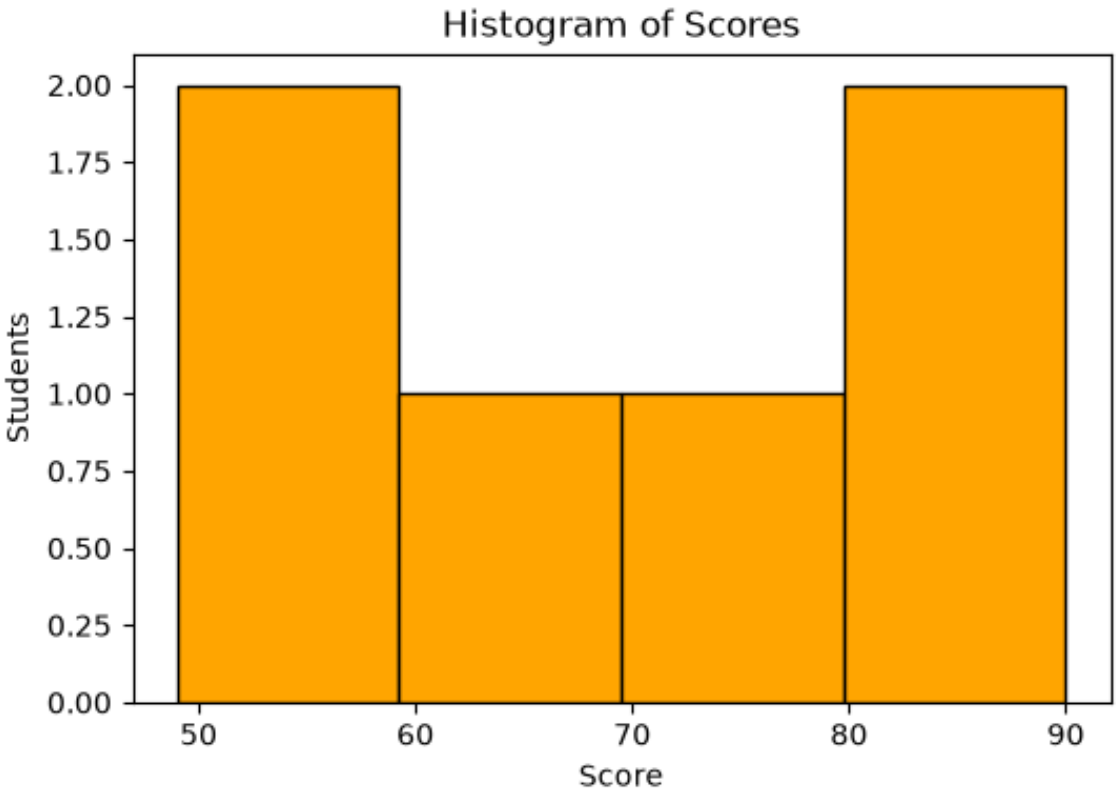


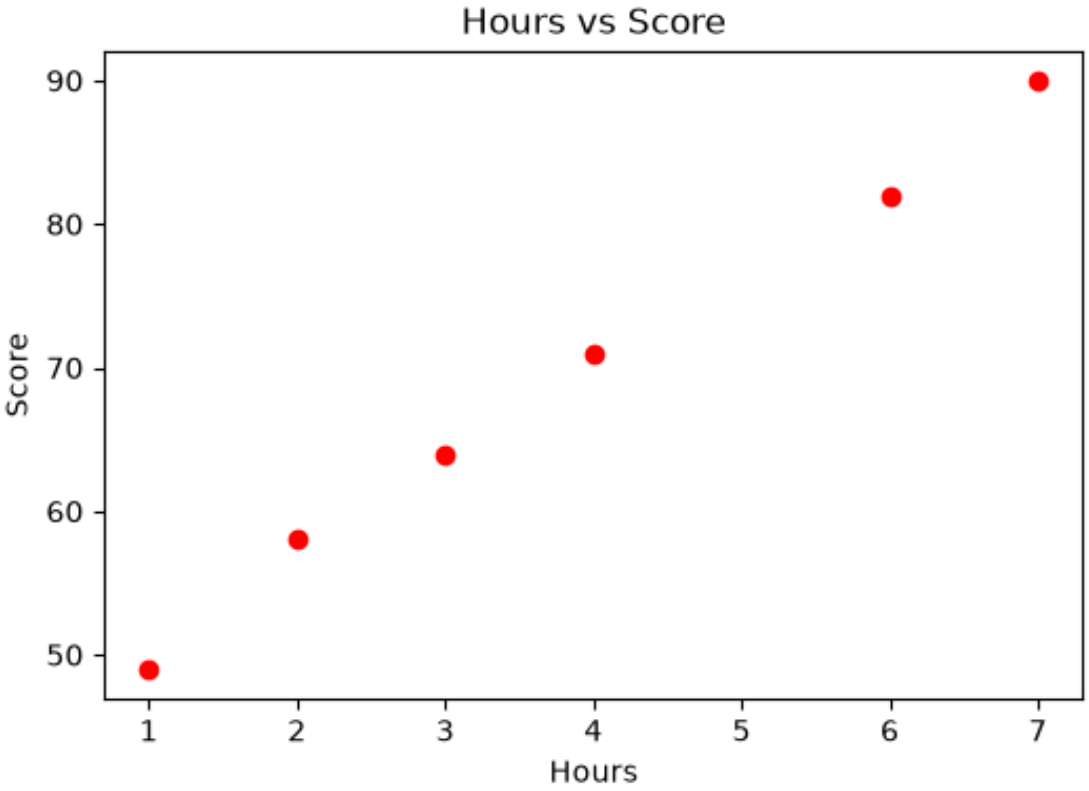


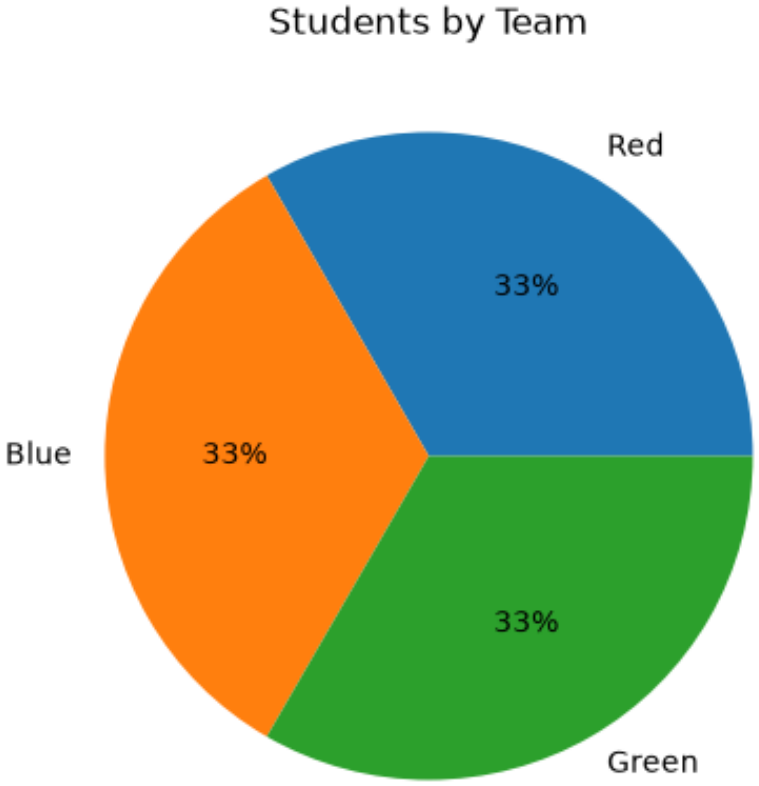
In [ ]:
```
plt.figure(figsize=(6, 4))
plt.hist(df["sc"], bins=4, color="orange", edgecolor="black")
plt.title("Histogram of Scores")
plt.xlabel("Score")
plt.ylabel("Students")
plt.show()

plt.figure(figsize=(6, 4))
plt.scatter(df["hr"], df["sc"], color="red")
plt.title("Hours vs Score")
plt.xlabel("Hours")
plt.ylabel("Score")
plt.show()

cn = df["tm"].value_counts()
plt.figure(figsize=(5, 5))
plt.pie(cn, labels=cn.index, autopct="%1.0f%%")
plt.title("Students by Team")
plt.show()
```



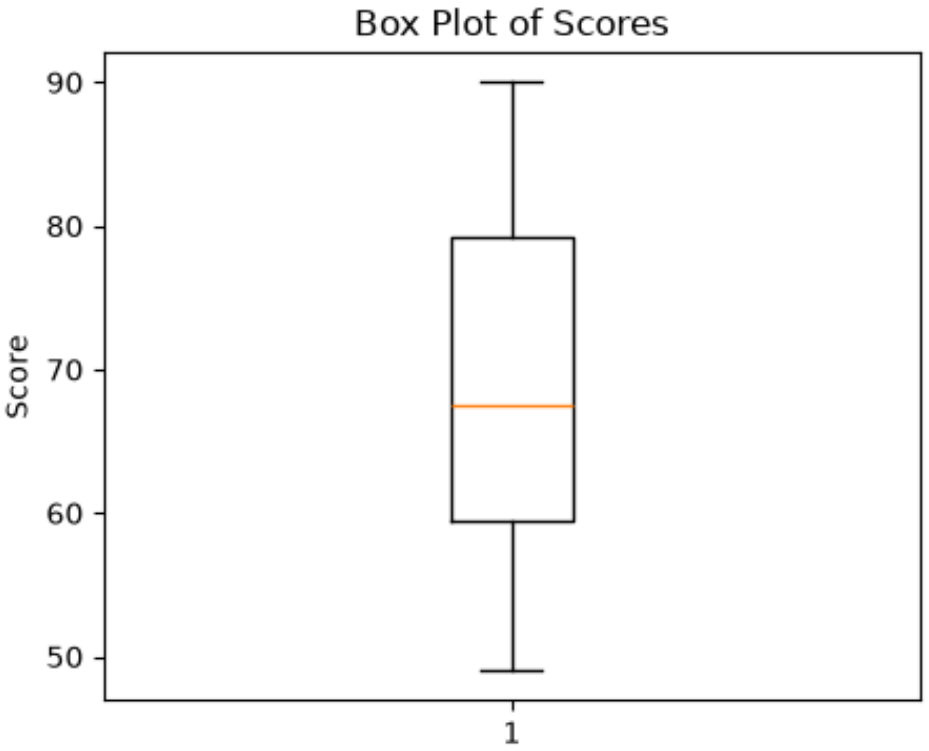


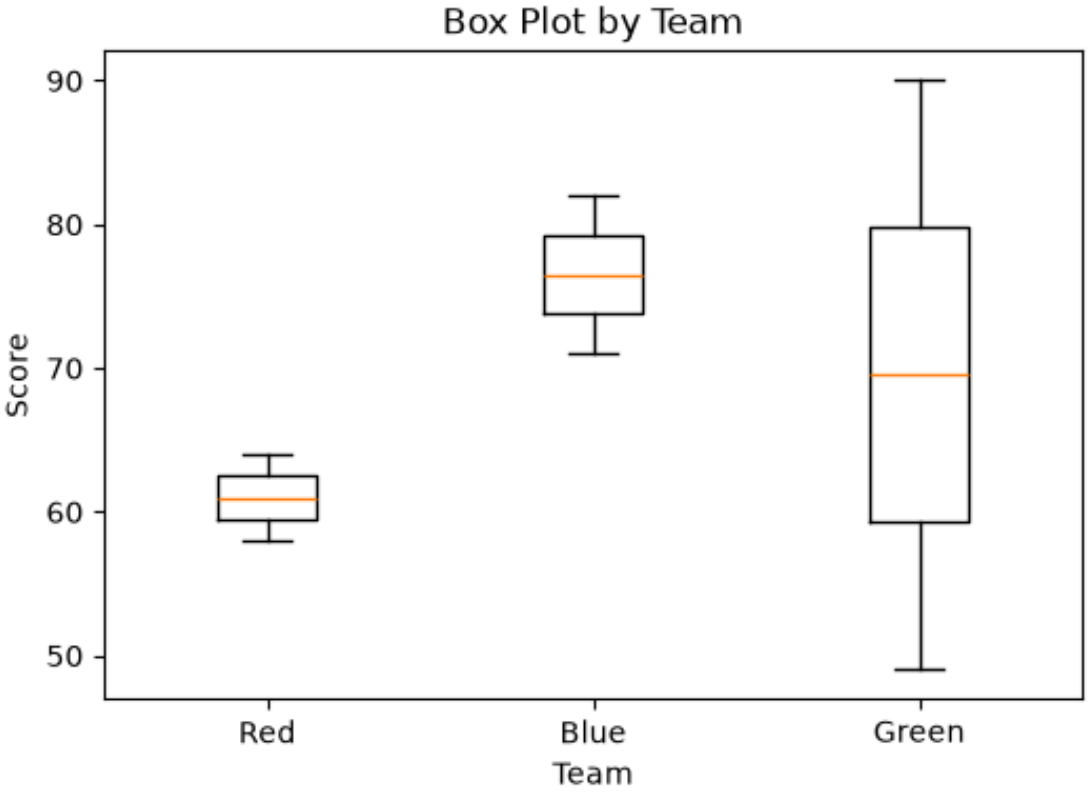


In [ ]:
```
plt.figure(figsize=(5, 4))
plt.boxplot(df["sc"])
plt.title("Box Plot of Scores")
plt.ylabel("Score")
plt.show()

rd = df[df["tm"] == "Red"]["sc"]
bl = df[df["tm"] == "Blue"]["sc"]
```

```
gr = df[df["tm"] == "Green"]["sc"]
plt.figure(figsize=(6, 4))
plt.boxplot([rd, bl, gr])
plt.xticks([1, 2, 3], ["Red", "Blue", "Green"])
plt.title("Box Plot by Team")
plt.xlabel("Team")
plt.ylabel("Score")
plt.show()
```



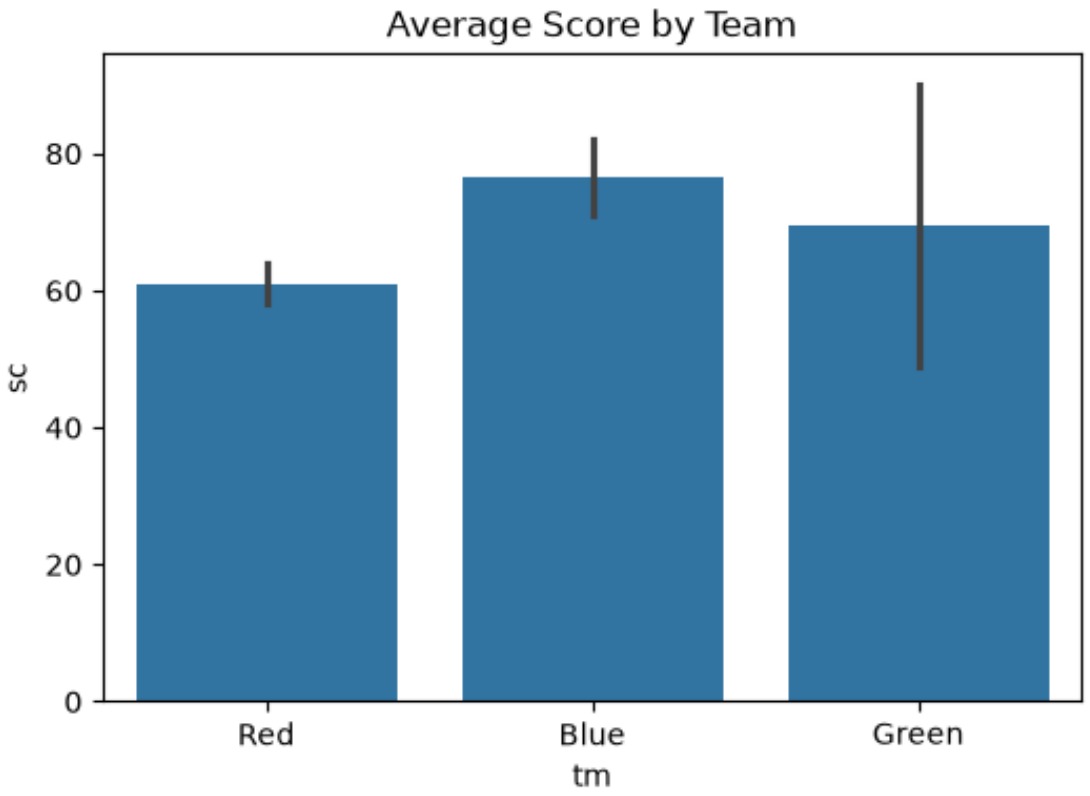


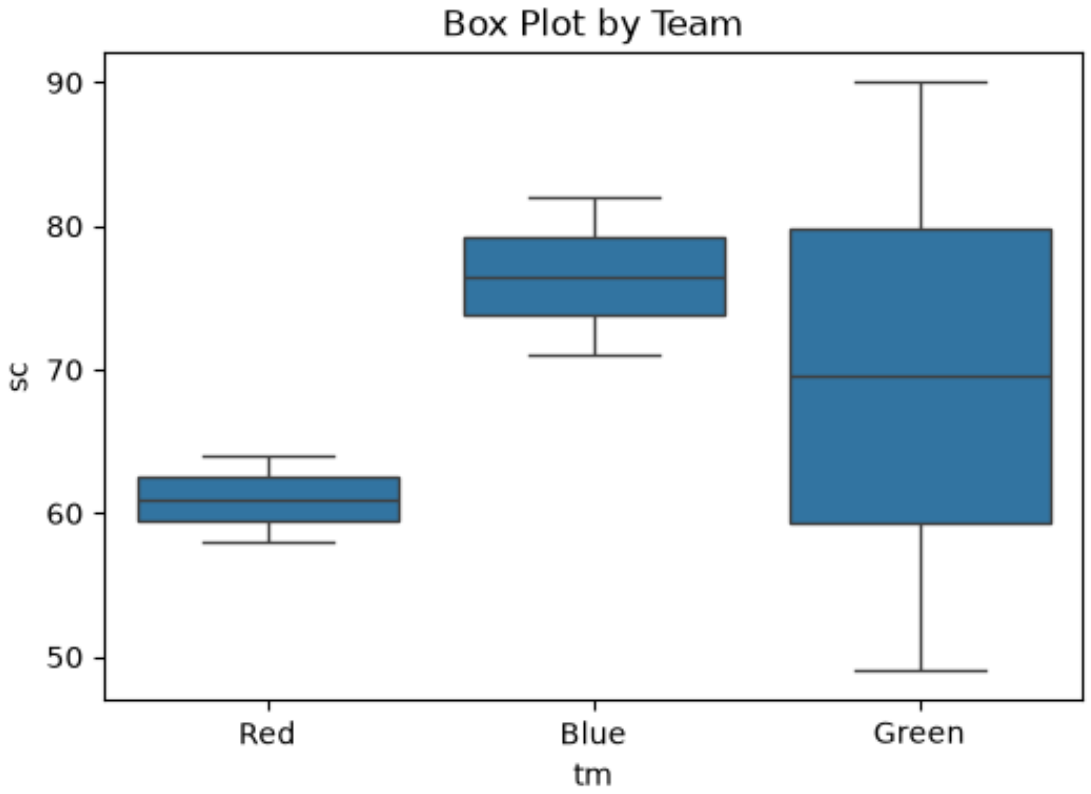
In [ ]:
```
plt.figure(figsize=(6, 4))
sns.barplot(x="tm", y="sc", data=df)
```

```
plt.title("Average Score by Team")
plt.show()

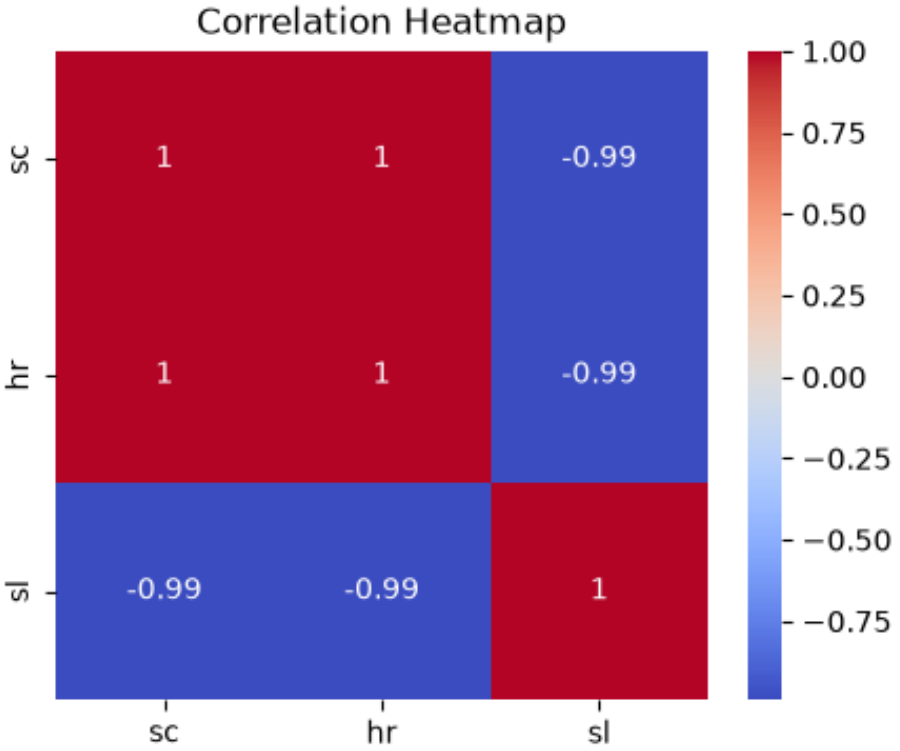
plt.figure(figsize=(6, 4))
sns.boxplot(x="tm", y="sc", data=df)
plt.title("Box Plot by Team")
plt.show()

cr = df[["sc", "hr", "sl"]].corr()
print(cr)
plt.figure(figsize=(5, 4))
sns.heatmap(cr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



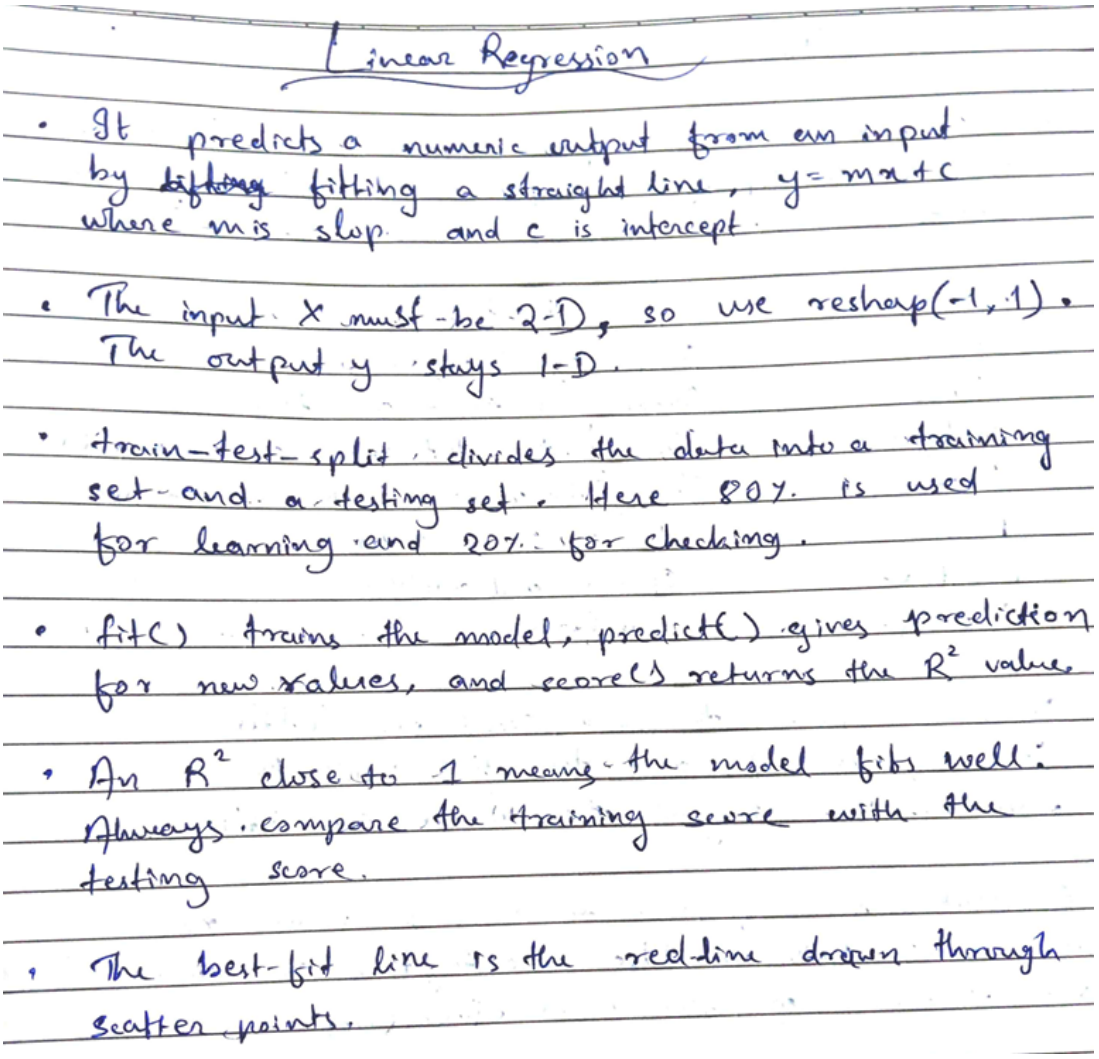


	sc	hr	sl
sc	1.000000	0.997577	-0.990177
hr	0.997577	1.000000	-0.987306
sl	-0.990177	-0.987306	1.000000



UNIT III

Linear Regression



Linear Regression

- It predicts a numeric output from an input by fitting a straight line, $y = mx + c$ where m is slop and c is intercept
- The input X must be 2-D, so use reshape(-1, 1). The output y stays 1-D.
- train_test_split divides the data into a training set and a testing set. Here 80% is used for learning and 20% for checking.
- fit() trains the model, predict() gives prediction for new values, and score() returns the R^2 value.
- An R^2 close to 1 means the model fits well. Always compare the training score with the testing score.
- The best-fit line is the red line drawn through scatter points.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

yr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
sl = np.array([30, 33, 38, 41, 47, 50, 55, 58, 64, 67])

x = yr.reshape(-1, 1)
y = sl
print("x shape:", x.shape)
print("y shape:", y.shape)

plt.figure(figsize=(6, 4))
plt.scatter(x, y, color="blue")
plt.title("Experience vs Salary")
plt.xlabel("Years")
plt.ylabel("Salary")
```

```
plt.show()
```

```
x shape: (10, 1)
y shape: (10,)
```



In [2]:
```
xtr, xte, ytr, yte = train_test_split(x, y, test_size=0.2, random_sta
print("Train samples:", len(xtr))
print("Test samples:", len(xte))

m = LinearRegression()
m.fit(xtr, ytr)
print("Model trained!")

sp = m.coef_[0]
ic = m.intercept_
print("Slope:", round(sp, 2))
print("Intercept:", round(ic, 2))
print("Equation: salary =", round(sp, 2), "* years +", round(ic, 2))

print("R2 train:", round(m.score(xtr, ytr), 3))
print("R2 test:", round(m.score(xte, yte), 3))
```

```
Train samples: 8
Test samples: 2
Model trained!
Slope: 4.24
Intercept: 24.97
Equation: salary = 4.24 * years + 24.97
R2 train: 0.996
R2 test: 0.999
```

In [3]:
```
p = m.predict([[7.5]])
print("Predicted salary for 7.5 years:", round(p[0], 1))

for h in [2, 5, 9]:
```

```
    p = m.predict([[h]])
    print("Years:", h, "-> Predicted salary:", round(p[0], 1))

plt.figure(figsize=(6, 4))
plt.scatter(x, y, color="blue", label="Actual data")
plt.plot(x, m.predict(x), color="red", label="Best fit line")
plt.title("Linear Regression")
plt.xlabel("Years")
plt.ylabel("Salary")
plt.legend()
plt.show()
```

```
Predicted salary for 7.5 years: 56.8
Years: 2 -> Predicted salary: 33.5
Years: 5 -> Predicted salary: 46.2
Years: 9 -> Predicted salary: 63.2
```

